# Connecting to MySQL from Python

Quick guide · mysql-connector-python · pandas + SQLAlchemy

In the SQL module you wrote queries in MySQL Workbench. In real projects the same queries are often run **from Python**: to pull data into pandas for analysis and charts, to load cleaned results back into the database, or to automate a report. This guide shows how to connect, create a table, insert data safely, run `SELECT`, `UPDATE` and `DELETE` queries, and move results into a pandas DataFrame.

Every example was run against a local MySQL-compatible server; the output shown is real.

| Tool | Use it to | Gives you |
|---|---|---|
| `mysql-connector-python` | run any SQL statement: create, insert, update, delete, select | rows as Python tuples or dictionaries |
| `pandas` + `SQLAlchemy` | read query results straight into a table; write a DataFrame to MySQL | a pandas DataFrame |

## Part 1 Setup and connecting

### Install the libraries

VS CODE TERMINAL

```
pip install mysql-connector-python pandas sqlalchemy
```

You also need a running MySQL server (the one MySQL Workbench connects to) and a username and password. Here the user is `student`.

### Keep the password out of your code

Never type a real password into a script you might share or push to GitHub. Store it in an **environment variable** and read it with `os.environ`:

| System | Set it in the terminal (current session) |
|---|---|
| Windows (PowerShell) | `$env:MYSQL_PASSWORD = "your_password"` |
| macOS / Linux / Git Bash | `export MYSQL_PASSWORD="your_password"` |

### Connect

m1_connect.py

```
import os
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="student",
    password=os.environ["MYSQL_PASSWORD"],
)
print("Connected:", conn.is_connected())

cursor = conn.cursor()
cursor.execute("CREATE DATABASE IF NOT EXISTS python_demo")
cursor.execute("SHOW DATABASES LIKE 'python%'")
print(cursor.fetchall())

cursor.close()
conn.close()
```

OUTPUT

```
Connected: True
[('python_demo',)]
```

| Object | Role |
|---|---|
| `conn` (connection) | the open line to the server; it owns `commit()`, `rollback()` and `close()` |
| `cursor` | sends one SQL statement at a time with `execute()` and fetches the results |

To work inside one database, add `database="python_demo"` to `connect()`, the same as `USE python_demo;` in Workbench.

## Part 2 Creating a table and inserting data

m2_create.py

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS branch_sales (
        sale_id     INT AUTO_INCREMENT PRIMARY KEY,
        sale_date   DATE NOT NULL,
        branch      VARCHAR(30) NOT NULL,
        product     VARCHAR(40) NOT NULL,
        units       INT NOT NULL,
        unit_price  DECIMAL(8, 2) NOT NULL
    )
""")
cursor.execute("SHOW TABLES")
print(cursor.fetchall())
```

OUTPUT

```
[('branch_sales',)]
```

### Insert one row, safely

m2_insert_one.py

```
sql = """INSERT INTO branch_sales (sale_date, branch, product, units, unit_price)
         VALUES (%s, %s, %s, %s, %s)"""
cursor.execute(sql, ("2026-10-01", "Sandton", "Coffee beans 1kg", 12, 189.99))
conn.commit()
print(cursor.rowcount, "row inserted; new id:", cursor.lastrowid)
```

OUTPUT

```
1 row inserted; new id: 1
```

> **Watch out:** Always pass values as the **second argument** with `%s` placeholders. Never build SQL by gluing strings together (`f"... WHERE branch = '{name}'"`): a malicious or simply odd value such as `O'Brien` can break or hijack the query (**SQL injection**). The connector escapes placeholder values for you.

Changes (`INSERT`, `UPDATE`, `DELETE`) are not saved until you call `conn.commit()`. Workbench commits automatically; Python does not.

### Insert many rows

m2_insert_many.py

```
rows = [
    ("2026-10-01", "Rosebank", "Coffee beans 1kg", 8, 189.99),
    ("2026-10-02", "Sandton", "Rooibos tea 80s", 25, 54.50),
    ("2026-10-02", "Soweto", "Coffee beans 1kg", 5, 179.99),
    ("2026-10-03", "Rosebank", "Rooibos tea 80s", 18, 54.50),
    ("2026-10-03", "Soweto", "Rusks 500g", 30, 42.00),
    ("2026-10-04", "Sandton", "Rusks 500g", 22, 44.00),
    ("2026-10-04", "Rosebank", "Rusks 500g", 15, 44.00),
]
cursor.executemany(sql, rows)
conn.commit()
print(cursor.rowcount, "rows inserted")
```

OUTPUT

```
7 rows inserted
```

## Part 3 Running SELECT queries

m3_select.py

```
cursor.execute("SELECT sale_id, sale_date, branch, product, units, unit_price FROM branch_sales")
rows = cursor.fetchall()
print(len(rows), "rows")
for row in rows[:3]:
    print(row)
```

OUTPUT

```
8 rows
(1, datetime.date(2026, 10, 1), 'Sandton', 'Coffee beans 1kg', 12, Decimal('189.99'))
(2, datetime.date(2026, 10, 1), 'Rosebank', 'Coffee beans 1kg', 8, Decimal('189.99'))
(3, datetime.date(2026, 10, 2), 'Sandton', 'Rooibos tea 80s', 25, Decimal('54.50'))
```

Each row is a **tuple**. MySQL types arrive as Python types: `DATE` becomes `datetime.date` and `DECIMAL` becomes `Decimal`, an exact decimal type used for money. Convert with `float()` if you need plain numbers.

| Fetch | Returns |
|---|---|
| `cursor.fetchall()` | a list of all remaining rows |
| `cursor.fetchone()` | the next row, or `None` when there are no more |
| `cursor.fetchmany(5)` | the next 5 rows, useful for large results |

## Rows as dictionaries

m3_dict.py

```
cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT branch, product, units FROM branch_sales ORDER BY units DESC LIMIT 3")
for row in cursor.fetchall():
    print(row["branch"], "|", row["product"], "|", row["units"])
```

OUTPUT

```
Soweto | Rusks 500g | 30
Sandton | Rooibos tea 80s | 25
Sandton | Rusks 500g | 22
```

`dictionary=True` lets you read fields by column name, as with the records in Session 09.

## Filtering with parameters

m3_where.py

```
branch = "Rosebank"
cursor.execute(
    "SELECT sale_date, product, units FROM branch_sales WHERE branch = %s AND units >= %s",
    (branch, 10),
)
for row in cursor.fetchall():
    print(row)
```

OUTPUT

```
(datetime.date(2026, 10, 3), 'Rooibos tea 80s', 18)
(datetime.date(2026, 10, 4), 'Rusks 500g', 15)
```

## Aggregating: GROUP BY

m3_groupby.py

```
cursor.execute("""
    SELECT branch,
           COUNT(*)                AS sales,
           SUM(units)              AS units,
           SUM(units * unit_price)  AS revenue
    FROM branch_sales
    GROUP BY branch
    ORDER BY revenue DESC
""")
print(cursor.column_names)
for branch, sales, units, revenue in cursor.fetchall():
    print(f"{branch:<10} {sales:>3} {units:>5} R{revenue:>10,.2f}")
```

OUTPUT

```
('branch', 'sales', 'units', 'revenue')
Sandton      3    59 R  4,610.38
Rosebank     3    41 R  3,160.92
Soweto       2    35 R  2,159.95
```

Any query from the SQL module works here unchanged: Python only sends the text and receives the rows. `cursor.column_names` gives the headings.

# Part 4 UPDATE, DELETE and handling errors

m4_update_delete.py

```
cursor.execute("UPDATE branch_sales SET unit_price = %s WHERE product = %s", (47.50, "Rusks 500g"))
print(cursor.rowcount, "rows updated")

cursor.execute("DELETE FROM branch_sales WHERE units < %s", (6,))
print(cursor.rowcount, "row deleted")

conn.commit()
cursor.execute("SELECT COUNT(*) FROM branch_sales")
print("Rows now:", cursor.fetchone()[0])
```

OUTPUT

```
3 rows updated
1 row deleted
Rows now: 7
```

`cursor.rowcount` tells you how many rows a statement changed: check it, just as you checked counts in the SQL workbook. Always use a `WHERE` clause; without one, `UPDATE` and `DELETE` touch every row.

## All or nothing: commit or roll back

m4_rollback.py

```
import os
import mysql.connector

conn = mysql.connector.connect(host="localhost", user="student",
                               password=os.environ["MYSQL_PASSWORD"], database="python_demo")
cursor = conn.cursor()
try:
    cursor.execute("UPDATE branch_sales SET units = units + 1 WHERE branch = %s", ("Sandton",))
    cursor.execute("UPDATE branch_sales SET unitz = 0")          # typo: column does not exist
    conn.commit()
except mysql.connector.Error as error:
    conn.rollback()
    print("Rolled back:", error)
finally:
    cursor.close()
    conn.close()
```

OUTPUT

```
Rolled back: 1054 (42S22): Unknown column 'unitz' in 'SET'
```

If any statement fails, `rollback()` undoes the earlier ones too, so the table is never left half-changed. `finally` closes the connection whether or not there was an error.

## Common connection errors

m4_errors.py

```
import mysql.connector

attempts = [
    {"user": "student", "password": "wrong-password"},
    {"user": "student", "password": "class2026", "database": "no_such_db"},
    {"user": "student", "password": "class2026", "port": 3307},
]
for settings in attempts:
    try:
        mysql.connector.connect(host="localhost", connection_timeout=3, **settings)
    except mysql.connector.Error as error:
        print(error.errno, "-", error.msg)
```

OUTPUT

```
1045 - Access denied for user 'student'@'localhost' (using password: YES)
1049 - Unknown database 'no_such_db'
2003 - Can't connect to MySQL server on 'localhost:3307' (Errno 111: Connection refused)
```

| Error number | Meaning | Check |
|---|---|---|
| 1045 | access denied | username and password |
| 1049 | unknown database | the database name; does it exist? |
| 2003 | cannot reach the server | is MySQL running? right host and port (default 3306)? |
| `ModuleNotFoundError` | library not installed | `pip install mysql-connector-python` in the same environment VS Code uses |

## Part 5 Query straight into pandas

For analysis, you usually want results as a DataFrame. pandas talks to MySQL through **SQLAlchemy**, using a connection URL:

m5_read_sql.py

```python
import os
import pandas as pd
from sqlalchemy import create_engine
from urllib.parse import quote_plus

password = quote_plus(os.environ["MYSQL_PASSWORD"])
engine = create_engine(f"mysql+mysqlconnector://student:{password}@localhost:3306/python_demo")

sales = pd.read_sql("SELECT * FROM branch_sales", engine, parse_dates=["sale_date"])
print(sales.dtypes)
print(sales)
```

OUTPUT

```
sale_id              int64
sale_date     datetime64[s]
branch                  str
product                 str
units                 int64
unit_price          float64
dtype: object
   sale_id  sale_date    branch           product  units  unit_price
0        1 2026-10-01   Sandton  Coffee beans 1kg     12      189.99
1        2 2026-10-01  Rosebank  Coffee beans 1kg      8      189.99
2        3 2026-10-02   Sandton   Rooibos tea 80s     25       54.50
3        5 2026-10-03  Rosebank   Rooibos tea 80s     18       54.50
4        6 2026-10-03    Soweto        Rusks 500g     30       47.50
5        7 2026-10-04   Sandton        Rusks 500g     22       47.50
6        8 2026-10-04  Rosebank        Rusks 500g     15       47.50
```

`quote_plus` makes passwords with characters such as `@` or `#` safe inside the URL. Without `parse_dates`, a `DATE` column arrives as plain Python dates (dtype `object`); `parse_dates=[...]` turns it into a real pandas datetime column ready for `.dt` and `resample`.

m5_params.py

```
query = """
    SELECT branch, product, SUM(units * unit_price) AS revenue
    FROM branch_sales
    WHERE sale_date >= %(start)s
    GROUP BY branch, product
"""
report = pd.read_sql(query, engine, params={"start": "2026-10-02"})
print(report.pivot_table(index="product", columns="branch", values="revenue",
fill_value=0).round(2))
```

OUTPUT

```
branch           Rosebank  Sandton  Soweto
product
Rooibos tea 80s     981.0   1362.5     0.0
Rusks 500g          712.5   1045.0  1425.0
```

Let SQL do the filtering and grouping on the server, and pandas do the reshaping, charts and analysis. Named placeholders `%(start)s` take a `params` dictionary.

### Write a DataFrame back to MySQL

m5_to_sql.py

```
targets = pd.DataFrame({"branch": ["Sandton", "Rosebank", "Soweto"],
                        "monthly_target": [25000, 20000, 15000]})
targets.to_sql("branch_targets", engine, if_exists="replace", index=False)

check = pd.read_sql("""
    SELECT t.branch, t.monthly_target, SUM(s.units * s.unit_price) AS revenue
    FROM branch_targets t
    LEFT JOIN branch_sales s ON s.branch = t.branch
    GROUP BY t.branch, t.monthly_target
""", engine)
print(check)
```

OUTPUT

```
     branch  monthly_target  revenue
0  Rosebank           20000  3213.42
1   Sandton           25000  4687.38
2    Soweto           15000  1425.00
```

`if_exists="replace"` drops and recreates the table; use `"append"` to add rows to an existing table.

## Part 6 Your sales_db from the SQL module

The same pattern works on the database built in the SQL sessions. Point the engine at `sales_db` and repeat the Day 2 quick checks from Python:

```
import os
import pandas as pd
from sqlalchemy import create_engine
from urllib.parse import quote_plus

engine = create_engine(

f"mysql+mysqlconnector://root:{quote_plus(os.environ['MYSQL_PASSWORD'])}@localhost:3306/sales_db")

counts = pd.read_sql("""
    SELECT 'customers' AS table_name, COUNT(*) AS row_count FROM customers
    UNION ALL SELECT 'products', COUNT(*) FROM products
    UNION ALL SELECT 'orders', COUNT(*) FROM orders
    UNION ALL SELECT 'order_items', COUNT(*) FROM order_items
""", engine)
print(counts)

by_region = pd.read_sql("""
    SELECT o.region, ROUND(SUM(oi.sales), 2) AS total_sales
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.order_id
    GROUP BY o.region
    ORDER BY total_sales DESC
""", engine)
by_region.plot.bar(x="region", y="total_sales", legend=False, title="Total sales by region")
```

Expected checks, as in the SQL workbook: **730** customers, **1,615** products, **1,980** orders and **4,000** order items. Use your own MySQL username in place of `root`. The last line draws the result as a chart, which is the step that is hard to do in Workbench.

## Try it yourself

TRY IT YOURSELF

Using `python_demo`: (a) insert two new sales for a **Midrand** branch with `executemany`; (b) query total units per **product**, largest first; (c) read the whole table into pandas and add a `revenue` column; (d) delete the Midrand rows and confirm with `rowcount`.

## Answers

m6_answers.py

```
import os
import mysql.connector
import pandas as pd
from sqlalchemy import create_engine
from urllib.parse import quote_plus

conn = mysql.connector.connect(host="localhost", user="student",
                               password=os.environ["MYSQL_PASSWORD"], database="python_demo")
cursor = conn.cursor()

cursor.executemany(
    "INSERT INTO branch_sales (sale_date, branch, product, units, unit_price) VALUES (%s, %s, %s,
%s, %s)",
    [("2026-10-05", "Midrand", "Coffee beans 1kg", 9, 189.99),
     ("2026-10-05", "Midrand", "Rusks 500g", 14, 47.50)])
conn.commit()
print("Inserted:", cursor.rowcount)

cursor.execute("SELECT product, SUM(units) AS units FROM branch_sales GROUP BY product ORDER BY
units DESC")
print(cursor.fetchall())

engine =
create_engine(f"mysql+mysqlconnector://student:{quote_plus(os.environ['MYSQL_PASSWORD'])}@localhost:
3306/python_demo")
df = pd.read_sql("SELECT * FROM branch_sales", engine)
df["revenue"] = df["units"] * df["unit_price"]
print(df[["branch", "product", "units", "revenue"]].tail(3))

cursor.execute("DELETE FROM branch_sales WHERE branch = %s", ("Midrand",))
conn.commit()
print("Deleted:", cursor.rowcount)
cursor.close()
conn.close()
```

OUTPUT

```
Inserted: 2
[('Rusks 500g', Decimal('81')), ('Rooibos tea 80s', Decimal('43')), ('Coffee beans 1kg',
Decimal('29'))]
     branch           product  units  revenue
6  Rosebank        Rusks 500g     15   712.50
7   Midrand  Coffee beans 1kg      9  1709.91
8   Midrand        Rusks 500g     14   665.00
Deleted: 2
```

## Checklist

| Habit | Why |
|---|---|
| Read the password from an environment variable | scripts get shared and pushed to GitHub |
| Use `%s` placeholders, never string formatting | prevents broken queries and SQL injection |
| `conn.commit()` after INSERT, UPDATE, DELETE | otherwise the change is lost |
| Check `cursor.rowcount` | confirms how many rows really changed |
| `try / except / rollback`, then `close()` in `finally` | no half-finished changes, no open connections |
| Filter and group in SQL, analyse in pandas | move less data; use each tool for its strength |